ASH

Chat that leaves no trace

Hackathon Project Documentation

A self-hostable, disposable, end-to-end encrypted peer-to-peer chat system built around WebRTC, ephemeral signaling, and local cryptography.

Project thesis

Messages should not have to live on a server just because two browsers want to talk. Ash uses a server only long enough to introduce peers. Once the direct connection exists, the conversation travels peer-to-peer and is encrypted before transmission.

1. Executive Summary

Ash is a disposable chat room designed around a simple security property: the infrastructure that helps people connect should not become the place where their conversation is stored.

A user creates a room and shares a link or QR code. Other users join without an account. A lightweight Go signaling server coordinates the WebRTC handshake using ephemeral in-memory state. After peers connect, messages travel over WebRTC DataChannels. Application-layer encryption using libsodium protects message content before it leaves the browser.

When participants leave, signaling state is destroyed. The intended architecture has no message database, account system, chat-history service, or server-side message archive.

Hackathon hook: the security model is demonstrable. Judges can inspect network behavior and verify that the signaling server is coordinating peers rather than storing the conversation.

Property	Ash's approach
Identity	No account; temporary room identity
Persistence	No chat database; room state is ephemeral
Transport	WebRTC DataChannel after signaling
Application security	libsodium-based E2EE layer
Infrastructure	Small Go signaling server; self-hostable
Lifecycle	Create -> connect -> chat -> leave -> destroy
Deployment	Local, VPS, or tunnel-assisted demo

2. Problem and Motivation

Traditional chat systems optimize for convenience: accounts, synchronization, history, backups, search, and server-side delivery. Those features also create persistent data and metadata.

Ash explores the opposite design space: a conversation that behaves like a temporary room rather than a permanent account.

Design goals

• **Ephemeral by default:** avoid persistent chat history and server-side message storage.

• **Private by construction:** encrypt content locally before sending it.

• **Peer-to-peer:** once connected, browsers exchange messages directly.

• **Simple deployment:** keep the backend small and self-hostable.

• **Inspectable:** make the architecture easy to demonstrate.

• **No account friction:** room links are the primary invitation mechanism.

Non-goals

• Ash is not a replacement for a full social messenger.

• It does not promise anonymity; network metadata can still exist outside the app.

• It cannot stop a recipient from copying or screenshotting a message.

• It is not a substitute for a professionally audited secure-messaging protocol.

3. User Experience and Demo Flow

The ideal demo should take less than five minutes and make the architecture visible rather than merely describing it.

1 - Create a room

Open Ash and select Create Room. The browser asks the signaling server for a room. The server generates an unpredictable room identifier and stores only temporary coordination state.

2 - Invite another peer

Ash shows a shareable link and optionally a QR code. A second browser/device opens it and joins without an account.

3 - Establish the peer connection

Peers exchange WebRTC setup information through the signaling server. The server relays setup data but is not the chat transport.

4 - Encrypt and chat

The sender encrypts plaintext locally, then sends an encrypted payload through the WebRTC DataChannel. The receiver decrypts locally and renders the plaintext.

5 - Prove it

Show signaling traffic and explain that it contains connection/setup information, not chat history. Demonstrate that application payloads are encrypted before transmission.

6 - Leave and destroy

On departure, the server removes room state. The intended design has no persistent message store from which the conversation can later be reconstructed.

Technical honesty: 'leaves no trace' is a product slogan, not a guarantee that no artifact exists anywhere. The precise claim is that Ash itself is designed not to persist chat content on its server.

4. System Architecture

Ash has four major layers: browser clients, ephemeral signaling, WebRTC peer-to-peer transport, and local cryptography.

4.1 Browser client

• Room creation/join UI and lifecycle controls.

• WebRTC peer connection and DataChannel management.

• Local key generation, key exchange, encryption, and decryption.

• Message rendering and local-only UI state.

• Optional QR-code invitation.

4.2 Go signaling server

• Creates and tracks temporary rooms.

• Tracks connected peers.

• Relays SDP and ICE signaling information.

• Handles join/leave events and cleanup.

• Does not require a message database.

4.3 WebRTC mesh

For N peers in a full mesh, there are N(N-1)/2 peer connections. This is excellent for a small-room demo but should be capped for scalability.

4.4 Cryptographic layer

Use a well-reviewed library rather than implementing cryptographic primitives manually. The proposed stack is libsodium-wrappers. A conceptual design can use X25519 for key agreement and an authenticated-encryption construction such as crypto_secretbox (XSalsa20-Poly1305).

Protocol caution: exact key management, peer authentication, replay protection, nonce handling, membership changes, and trust assumptions must be specified before claiming production-grade E2EE.

Architecture diagram - PlantUML

```
@startuml Ash_Architecture
actor "Peer A\n(Browser)" as A
actor "Peer B\n(Browser)" as B
node "Self-Hosted Host" {
  component "Go Signaling Server\n(in-memory only)" as S
  component "Ephemeral Room State" as R
}
cloud "Optional Tunnel" as T
package "Browser Crypto" {
  component "X25519 Key Agreement" as K
  component "Authenticated Encryption\n(XSalsa20-Poly1305)" as E
}
A --> T : HTTPS / WSS
B --> T : HTTPS / WSS
T --> S : forward
S --> R : room + peer state
S ..> A : SDP / ICE
S ..> B : SDP / ICE
A <..> B : WebRTC DataChannel\nDirect P2P
A --> K
A --> E
B --> K
B --> E
@enduml
```

5. Detailed Technical Flows

5.1 Room creation

• Client sends a create-room request.

• Server creates a cryptographically random room ID.

• Server stores temporary membership and connection state.

• Client receives the room link.

• No chat message is created or stored.

5.2 Peer joining

• Joining browser connects to signaling.

• Server verifies the room and adds the new peer.

• Existing peers receive a peer-joined event.

• WebRTC offer/answer and ICE candidates are exchanged.

• Peers establish DataChannels.

5.3 Message transmission

• User enters plaintext locally.

• Client selects the appropriate key.

• Client generates a fresh nonce as required by the API.

• Plaintext is authenticated/encrypted locally.

• Ciphertext and protocol metadata are sent over the DataChannel.

• Receiver verifies/decrypts locally.

5.4 Room teardown

• Client leaves or disconnects.

• Peer is removed from memory.

• Empty room state is deleted.

• No message history is persisted.

Sequence diagram - PlantUML

```
@startuml Ash_Message_Flow
participant "Peer A" as A
participant "Signaling Server" as S
participant "Peer B" as B
A -> S : Create room
S -> S : Generate room ID
S --> A : Room link
A -> B : Share link
B -> S : Join room
S -> A : Peer joined
A -> S : SDP / ICE
S -> B : Relay signaling data
B -> S : SDP / ICE
S -> A : Relay signaling data
A <-> B : WebRTC DataChannel
loop Chat
  A -> A : Encrypt locally
  A -> B : Ciphertext
  B -> B : Verify + decrypt locally
end
A -> S : Leave
```

```
B -> S : Leave
S -> S : Delete empty room state
@enduml
```

6. Backend Design

The signaling backend should intentionally be boring. Its purpose is coordination, not storage or message processing.

Component	Responsibility
HTTP/WebSocket handler	Room creation, joins, signaling, lifecycle events
Room manager	Map room IDs to temporary room state
Peer registry	Track peer IDs and signaling channels
Cleanup	Delete peers/rooms on disconnect; optional TTLs
Health endpoint	Minimal liveness endpoint
Logging	Operational logs without message content

Suggested in-memory model

```
rooms map[string]*Room

Room
  ID
  createdAt
  peers map[string]*Peer

Peer
  ID
  signalingConnection
  publicKey / protocol metadata
  connection state
```

Server rules

• Never expose a persistent send-message API.

• Never write plaintext messages to logs.

• Avoid persistent analytics tied to room IDs.

• Use short-lived room IDs and expiration.

• Treat all client-supplied signaling data as untrusted input.

7. Security and Cryptography Model

The privacy story depends on separating signaling from message transport. The server needs enough information to introduce peers, but should not need plaintext conversation data.

Conceptual message path

```
plaintext
   |
   v
[local client]
   |
   | authenticated encryption
   v
ciphertext + nonce + metadata
   |
   v
WebRTC DataChannel
   |
   v
[receiving client]
   |
   | verify + decrypt
   v
plaintext
```

Key-management questions

• **Key authentication:** how does a client know a public key belongs to the intended peer?

• **Group membership:** how are keys distributed when peers join or leave?

• **Nonce safety:** how is a unique nonce generated for every encryption operation?

• **Replay protection:** what prevents an old ciphertext from being accepted again?

• **Key rotation:** what changes when a participant leaves?

• **Trust model:** exactly which attackers is Ash intended to resist?

Recommended hackathon posture

Do not invent cryptography. Use established libsodium APIs and keep the protocol small. If strong group key management cannot be completed during the hackathon, narrow the MVP and document the limitation instead of making an exaggerated security claim.

8. Networking and Deployment

WebRTC enables browser-to-browser communication, but peers can be behind NATs or firewalls. Ash therefore needs signaling and ICE configuration even though it aims for direct peer transport.

STUN/TURN

• **STUN:** helps peers discover usable network paths.

• **TURN:** relays traffic when direct connectivity is impossible; such sessions are not network-direct P2P.

• **Application E2EE:** ciphertext can remain protected through a relay if the cryptographic protocol is correct.

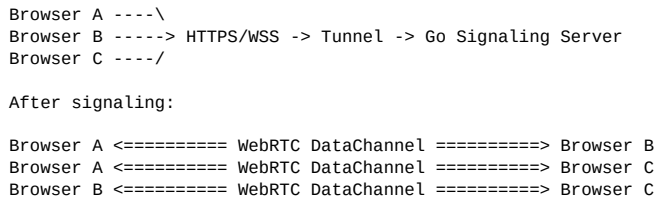
Self-hosting

Package the Go server into Docker. Run it on a laptop, VPS, or small cloud instance. During the hackathon, an outbound tunnel can expose a local server publicly without opening an inbound firewall port.

Tunnel caveat

A tunnel is not the privacy boundary for Ash's messages. Application-layer E2EE protects message content. The tunnel/provider may still observe connection-level metadata depending on configuration and infrastructure.

Deployment topology



9. Technology Stack

Layer	Technology	Why
Frontend	React + Vite	Fast iteration and componentized UI
Browser networking	WebRTC	Native browser P2P transport
Signaling backend	Go	Small, concurrent, self-hostable
Realtime signaling	WebSocket	Natural SDP/ICE event transport
Cryptography	libsodium-wrappers	Established primitives for JavaScript
Deployment	Docker	Reproducible deployment
Optional exposure	Cloudflare Tunnel	Convenient public demo access
Optional invite UX	QR code	Fast device-to-device joining

10. Suggested Repository Structure

```
ash/
├── client/
│   ├── src/
│   │   ├── components/
│   │   ├── crypto/
│   │   ├── webrtc/
│   │   ├── signaling/
│   │   ├── room/
│   │   └── app/
│   └── package.json
├── server/
│   ├── cmd/ash/
│   ├── internal/
│   │   ├── signaling/
│   │   ├── rooms/
│   │   └── peers/
│   ├── go.mod
│   └── Dockerfile
├── docs/
│   ├── architecture.puml
│   └── protocol.md
├── docker-compose.yml
└── README.md
```

11. Hackathon MVP and Roadmap

Build order matters. The first objective is a working end-to-end demo, not a huge feature list.

Phase	Deliverable	Priority
1	React/Vite shell + create/join UI	Must have
2	Go signaling + WebSocket protocol	Must have
3	Two-browser WebRTC DataChannel	Must have
4	Local encryption/decryption	Must have
5	Room cleanup + no persistence	Must have
6	Polished UI + connection state	Should have
7	QR invite + copy-link UX	Should have
8	Docker + public deployment	Should have
9	Encrypted file transfer	Stretch
10	Panic-close / emergency destruction	Stretch
11	Advanced group key management	Stretch / advanced

Judge-facing demo script

• **0:00-0:30:** Explain the problem: normal chat assumes persistent infrastructure.

• **0:30-1:00:** Create a room with no account and show the share link/QR.

• **1:00-2:00:** Join from a second browser/device and show the peer connection.

• **2:00-3:00:** Send messages and explain local encryption before DataChannel transmission.

• **3:00-4:00:** Show the signaling architecture and ephemeral room state.

• **4:00-5:00:** Close the room and show teardown/self-hosting.

12. Why Ash Is a Strong Hackathon Project

Ash is more than a chat UI: the interesting part is the systems design underneath. It combines browser networking, distributed state, cryptographic protocol design, NAT traversal, real-time signaling, ephemeral infrastructure, and a highly demonstrable user experience.

Judging differentiators

• **Visually simple, technically deep.** Minimal interface, sophisticated systems underneath.

• **Easy to understand.** 'Chat without storing the chat' is an immediate pitch.

• **Live proof.** Demonstrate network behavior and lifecycle rather than only slides.

• **Self-hostable.** Judges can understand what infrastructure is under the team's control.

• **Expandable.** File transfer, voice/video, temporary collaboration rooms, and stronger cryptography can build on the foundation.

Potential objections and answers

Judge asks	Answer
Why use a server at all?	Browsers still need signaling to discover and negotiate peers. Ash keeps that service ephemeral and out of
What if the server is compromised?	The intended design keeps plaintext out of signaling; confidentiality depends on correct E2EE and key auth
What if P2P fails?	ICE may use a TURN relay; application E2EE can still protect message content.
Can users save messages?	They can copy or screenshot anything they can see. Ash's guarantee concerns server-side persistence.
Is it anonymous?	No. Ash is privacy-oriented, not an anonymity network.
Can a recipient retain messages?	Yes. E2EE does not control what a legitimate recipient does with received content.

13. Risks, Limitations, and Mitigations

Risk	Impact	Mitigation
Full mesh does not scale	High	Cap room size; consider SFU later.
NAT/firewall blocks direct P2P	Medium	Configure STUN/TURN and communicate relay behavior.
Weak key authentication	High	Define a trust model and authenticated key exchange.
Nonce/key misuse	High	Use libsodium APIs correctly; centralize crypto code.
Accidental logging	Medium	Redact payloads and avoid message content in logs.
Abandoned rooms	Low/Medium	Disconnect handling and TTL garbage collection.
Browser reconnects	Medium	Rebuild peer state safely on reconnect.
Overclaiming privacy	High	Claim no server-side chat persistence, not zero trace everywhere.

14. Future Extensions

• **Ephemeral file transfer:** encrypted file chunks over DataChannels.

• **Voice/video:** WebRTC media tracks with the same room lifecycle.

• **Stronger group cryptography:** formal group key management, rotation, and membership semantics.

• **Room policies:** expiry timers, participant limits, invite approval, host controls.

• **Local-only export:** explicitly export a user's conversation without server persistence.

• **Auditable protocol:** publish a compact protocol specification and threat model.

15. One-Line Pitch

Ash is a disposable, self-hostable P2P chat room where the server introduces you, but never needs to own your conversation.

16. Final Architecture Principle

Think of Ash as **a matchmaker, not a mailbox.** The server gets peers connected and then gets out of the way. Browsers own the conversation, the cryptographic layer protects content, and room lifecycle makes persistence an exception rather than the default.

Prepared as a hackathon project concept and technical planning document.